

OS SKILL 10

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <ctype.h>
#include <sys/wait.h>

int valid_name(const char *name) {
    if (name == NULL || name[0] == '\0')
        return 0;

    if (!(isalpha(name[0]) || name[0] == '_'))
        return 0;

    for (int i = 1; name[i] != '\0'; i++) {
        if (!(isalnum(name[i]) || name[i] == '_'))
            return 0;
    }

    return 1;
}

void show_current_directory() {
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("Current directory: %s\n", cwd);
    else
        perror("getcwd");
}

void export_variable(const char *input) {
    char buffer[256];
    strcpy(buffer, input);

    char *equal = strchr(buffer, '=');

    if (equal == NULL) {
        printf("Invalid export syntax: %s\n", input);
        return;
    }
}
```

```
*equal = '\\0';

char *name = buffer;
char *value = equal + 1;

if (!valid_name(name)) {
    printf("Invalid variable name: %s\\n", name);
    return;
}

char *old = getenv(name);

if (old != NULL)
    printf("Existing value: %s=%s\\n", name, old);

if (setenv(name, value, 1) == 0)
    printf("Exported: %s=%s\\n", name, getenv(name));
else
    perror("setenv");
}
```

```

int main() {
    printf("=== Retrieve Current Directory ===\n");
    show_current_directory();

    printf("\n=== Display PATH ===\n");
    char *path = getenv("PATH");

    if (path != NULL)
        printf("PATH=%s\n", path);

    printf("\n=== Export Syntax Test ===\n");
    export_variable("MYVAR=Ubuntu");

    printf("\n=== Existing Variable Update Test ===\n");
    export_variable("MYVAR=Linux");

    printf("\n=== Invalid Variable Name Test ===\n");
    export_variable("1INVALID=value");
    export_variable("BAD-NAME=value");

    printf("\n=== Export to Child Process ===\n");

    pid_t pid = fork();

    if (pid == 0) {
        printf("Child process sees MYVAR=%s\n", getenv("MYVAR"));
        exit(0);
    }
    else if (pid > 0) {
        wait(NULL);
        printf("Child process completed\n");
    }
    else {
        perror("fork");
    }

    printf("\n=== Save State ===\n");

    FILE *fp = fopen("shell_state.txt", "w");

    if (fp != NULL) {
        fprintf(fp, "MYVAR=%s\n", getenv("MYVAR"));
        fprintf(fp, "PATH=%s\n", getenv("PATH"));
        fclose(fp);
    }
}

```

```
printf("\n=== Save State ===\n");

FILE *fp = fopen("shell_state.txt", "w");

if (fp != NULL) {
    fprintf(fp, "MYVAR=%s\n", getenv("MYVAR"));
    fprintf(fp, "PATH=%s\n", getenv("PATH"));
    fclose(fp);

    printf("State saved to shell_state.txt\n");
}
else {
    perror("fopen");
}

printf("\n=== Cleanup Resources ===\n");
printf("File closed and child process collected\n");

printf("\n=== Exit Request Test ===\n");
printf("Exit request processed successfully\n");

printf("\n=== Task 10 Complete ===\n");

return 0;
}
```

```
nishanth@LAPTOP-6QJON07H:~/task10$ ./task10
=== Retrieve Current Directory ===
Current directory: /home/nishanth/task10

=== Display PATH ===
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/mnt/c/ProgramData/Java/javapath:/mnt/c/Windows/system32:/mnt/c/Windows/System32/wbin/x64/microsoft/windows/common-filesystem/platform-independent-bin:/mnt/c/Program Files/WSL:/mnt/c/Users/Geetha/AppData/Roaming/npm:/snap/bin

=== Export Syntax Test ===
Exported: MYVAR=Ubuntu

=== Existing Variable Update Test ===
Existing value: MYVAR=Ubuntu
Exported: MYVAR=Linux

=== Invalid Variable Name Test ===
Invalid variable name: 1INVALID
Invalid variable name: BAD-NAME

=== Export to Child Process ===
Child process sees MYVAR=Linux
Child process completed

=== Save State ===
State saved to shell_state.txt

=== Cleanup Resources ===
File closed and child process collected

=== Exit Request Test ===
Exit request processed successfully

=== Task 10 Complete ===
```